

Feuille de travaux pratiques n° 4

Mémoire transactionnelle logicielle — CORRECTION —

Code de départ associé : <https://github.com/DistributedComputing/TP-transactions>.

1 Introduction

Le concept de mémoire transactionnelle a été proposé par Herlihy et Moss en 1993 [1] sur le modèle des transactions en bases de données. L'algorithme TL2 a été proposé par Dice, Shalev et Shavit en 2006 [2]. La présentation de l'algorithme et la formulation de ce sujet ont été largement inspirées d'un article de Imbs et Raynal de 2009 [3].

L'idée est la suivante : comme avec l'utilisation de verrous, le code à exécuter atomiquement doit être encadré par l'appel de deux fonctions, `begin` et `try_to_commit`. Entre les deux, il est possible de faire des lectures et des écritures sur des registres X en appelant les fonctions `X.get()` et `X.set(v)`.

Les lectures, les écritures et l'invocation de la fonction `try_to_commit` peuvent *aborter*. Dans ce cas, on dit que la transaction complète aborte. Intuitivement, une transaction abortée n'a aucun effet et doit être redémarrée. Le critère de cohérence attendu sur les transactions est appelé *sérialisabilité*. Une exécution est sérialisable si elle est équivalente à une exécution séquentielle (c'est-à-dire dans laquelle il n'y a pas de concurrence entre les transactions) qui contient toutes (et ne contient que) les transactions qui n'ont pas aborté. Autrement dit, l'exécution dans laquelle toutes les transactions abortées ont été supprimées, est séquentiellement cohérente.

La mémoire transactionnelle fournit deux interfaces : l'interface `Transaction` déclare les méthodes `begin` et `try_to_commit`, et l'interface `Register` fournit les méthodes permettant de lire et écrire dans un registre. Les aborts sont ici gérés à l'aide d'exceptions de type `AbortException`.

Le code suivant présente un exemple d'utilisation de la mémoire transactionnelle. Il s'agit essentiellement d'une transaction contenant deux lectures et deux écritures, au sein d'une boucle réessayant d'exécuter la transaction jusqu'à ce qu'elle soit acceptée.

```
void echange (Register<Integer> X, Register<Integer> Y) {  
    Transaction t = new STMTransaction();  
    for (;;) {  
        try {  
            t.begin();  
            Integer x = X.get(t);  
            X.set(Y.get(t), t);  
            Y.set(x, t);  
            t.try_to_commit();  
            return;  
        } catch (AbortException e) {}  
    }  
}
```

Algorithm 1: Algorithme TL2

```

1 operation  $T.begin()$ :
2   re-initialize local variables
3    $birthDate \leftarrow clock$ 
4 operation  $X.set(T, v)$ :
5   if there is no local copy  $lcx$  of  $X$  :
6     allocate local space  $lcx$  for a copy
7    $lcx.value \leftarrow v$ ;  $lws_T \leftarrow lws_T \cup \{X\}$ 
8 operation  $X.get(T)$ :
9   if there is a local copy  $lcx$  of  $X$  :
10    return  $lcx.value$ 
11  else:
12     $lcx \leftarrow X.copy$ ;  $lrs_T \leftarrow lrs_T \cup \{X\}$ 
13    if  $lcx.date > birthDate$  : abort
14    else: return  $lcx.value$ 
15 operation  $T.try\_to\_commit()$ :
16   lock all the objects in  $lws_T$ 
17   if some object in  $lrs_T$  is already locked :
18     release all the locks; abort
19   for each  $X \in lrs_T$  :
20     if  $X.date > birthDate$  :
21       release all the locks; abort
22    $commitDate \leftarrow clock.getAndIncrement()$ 
23   for each  $X \in lws_T$  :
24      $X \leftarrow (lcx.value, commitDate)$ 
25   release all the locks;

```

2 L'algorithme *Transactional Locking 2*

Beaucoup d'algorithmes ont été proposés pour implémenter la mémoire transactionnelle, dont les plus simples sont expliqués par l'article de Raynal et Imbs [3]. La lecture de cet article, en annexe de ce sujet, est vivement recommandée. L'algorithme 1 présente Transactional Locking 2 (TL2). Il est basé sur une horloge logique globale, *clock*, incrémentée à chaque fois qu'une transaction est sur le point d'être acceptée (ligne 22), et qui décrit l'ordre (ou plutôt *un* ordre) dans laquelle les transactions acceptées sont sérialisées.

Chaque registre X accessible dans une transaction comporte deux champs : un champ de données $X.value$ qui contient la dernière valeur écrite et un champ de contrôle $X.date$ qui contient la date de la dernière transaction acceptée à avoir écrit dans X . De plus, un verrou est associé à chaque registre. Chaque transaction T gère une variable locale lrs_T contenant l'ensemble des variables lues, une variable locale lws_T contenant l'ensemble des variables écrites au cours de T et une variable *birthDate* contenant la valeur de *clock* au début de la transaction.

Lors d'une écriture sur X , une copie locale de X est créée pour conserver la valeur écrite. De plus X est placée dans l'ensemble lws_T . Lors d'une lecture de X , si X a été précédemment écrite par la même transaction, la valeur locale est retournée. Sinon, X est placée dans l'ensemble lrs_T , puis il est vérifié que X n'a pas été modifiée depuis le début de la transaction en comparant la date de X à *birthDate* (auquel cas l'abort est nécessaire) et la valeur en mémoire partagée est retournée.

La fonction *try_to_commit* est bloquante. Lorsqu'elle est appelée, la transaction prend les verrous sur tous les registres puis vérifie que les valeurs lues sont toujours cohérentes entre elles, et cohérentes avec les nouvelles valeurs qu'elle a écrites localement. Cela se fait en comparant la date actuelle de chaque registre lu à la date *birthDate* du début de la transaction. Si l'une de ces dates est plus grande, un registre a été écrit par une transaction concurrente et un abort est nécessaire. Sinon, la transaction peut être acceptée : les écritures sont reportées sur la mémoire partagée, à une date obtenue en incrémentant l'horloge globale.

Récupérez le code de départ dans le dépôt GitHub. Votre travail consiste à implémenter une mémoire transactionnelle logicielle fonctionnant selon l'algorithme TL2. Chaque fonction devra être starvation-free.

3 Références

- [1] Maurice Herlihy et J. Eliot B. Moss. *Transactional memory : architectural support for lock-free data structures*. International Symposium on Computer Architecture (1993)
- [2] Dave Dice, Ori Shalev et Nir Shavit. *Transactional Locking II*. International Symposium on Distributed Computing (2006)
- [3] Damien Imbs et Michel Raynal. *Software transactional memories : an approach for multicore programming*. International Conference on Parallel Computing Technologies (2009)

De nombreuses implémentations des mémoires transactionnelles logicielles sont disponibles en Java, même si aucune n'est universellement acceptée. Voici des liens vers quelques unes des plus connues.

AtomJava : https://wasp.cs.washington.edu/wasp_atomjava.html

JVSTM : <http://inesc-id-esw.github.io/jvstm>

Deuce STM : <https://sites.google.com/site/deucestm>

Multiverse : <https://github.com/pveentjer/Multiverse>

DSTM2 : <http://www.oracle.com/technetwork/indexes/downloads/index.html>

ObjectFabric : <https://github.com/cypof/objectfabric>

Étant données les limitations des mémoires transactionnelles logicielles, beaucoup de chercheurs militent pour l'introduction de mémoires transactionnelles matérielles, dans lesquelles les mêmes propriétés sont assurées par l'architecture physique de la mémoire partagée, au même titre que les caches.